AADK TASK 2 Compose Essentials: Build, Analyze, Refine

Android Studio Setup and First Compose App

Name: Vismaya T M

TASK 1

Setup Process

I installed Android Studio successfully and created a new project using Empty Compose Activity. Gradle sync completed without errors. I used Kotlin as the programming language and Jetpack Compose for UI development.

Device Configuration

I enabled Developer Options on my OPPO A3 Pro 5G by tapping Version Number multiple times. Then I enabled USB Debugging. After connecting the phone via USB and selecting File Transfer mode, the device was detected in Android Studio.

Running the App

I clicked the Run button in Android Studio. The APK was installed successfully, and the Greeting Card application launched on my physical device.

1. What does setContent {} do?

The setContent {} function is used to define the UI of the application using Jetpack Compose. It replaces the traditional XML layout system and allows UI to be written directly in Kotlin.

2. What does @Composable mean?

@Composable is an annotation used to define functions that build UI components in Jetpack Compose.

Lessons Learned

1. Understanding Android Studio Setup

I learned how to install and configure Android Studio properly, including SDK setup and Gradle synchronization.

2. Importance of Gradle Sync

I understood that Gradle sync is essential for building the project, and it may take time during the first setup.

3. Working with Jetpack Compose

I learned how UI can be created using Jetpack Compose instead of XML and how composable functions build the screen.

4. Role of setContent and @Composable

I understood that setContent {} defines the app's UI in Compose and @Composable functions are used to create reusable UI components.

5. Running App on Physical Device

- Enable Developer Options
- Enable USB Debugging
- Change USB mode from Charging to File Transfer
- Connect my OPPO device successfully

6. Importance of Rebuilding the App

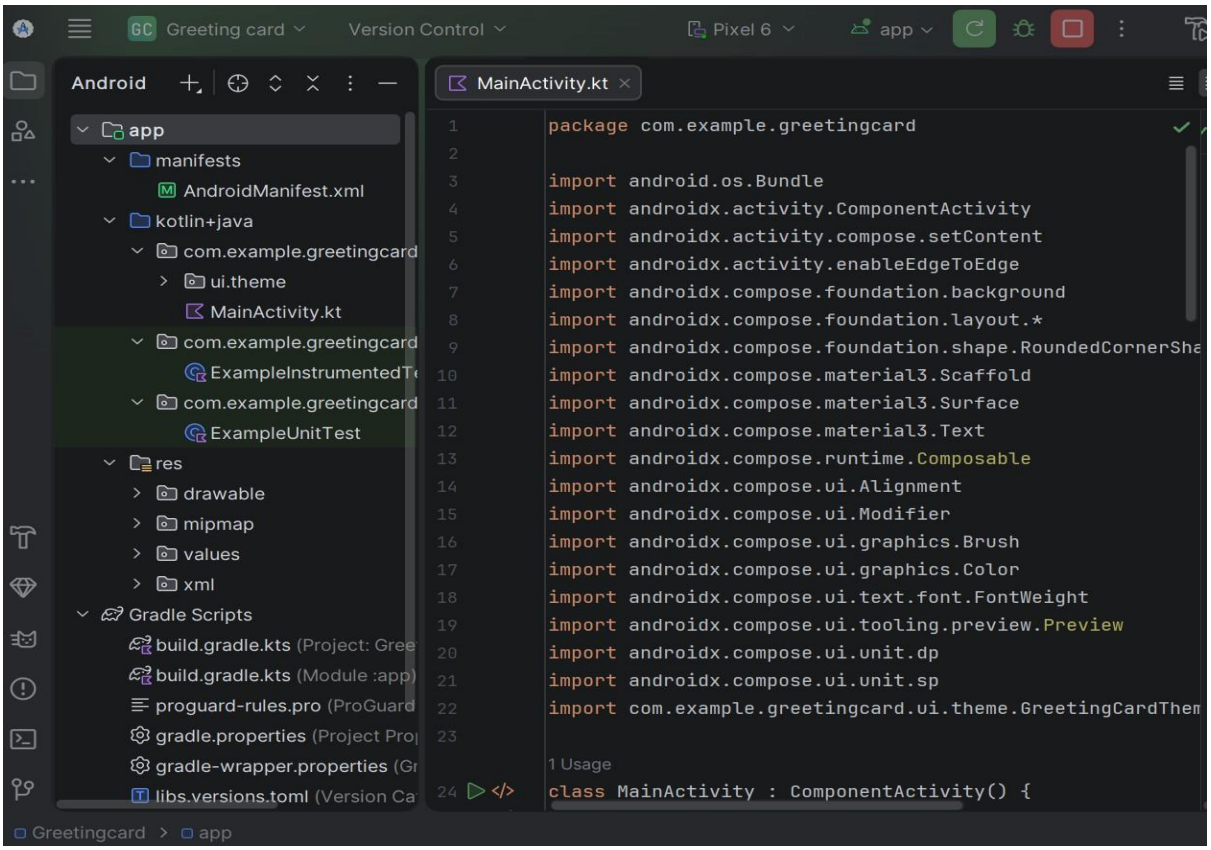
I learned that after making UI changes, I must re-run the app to see updated results on the device.

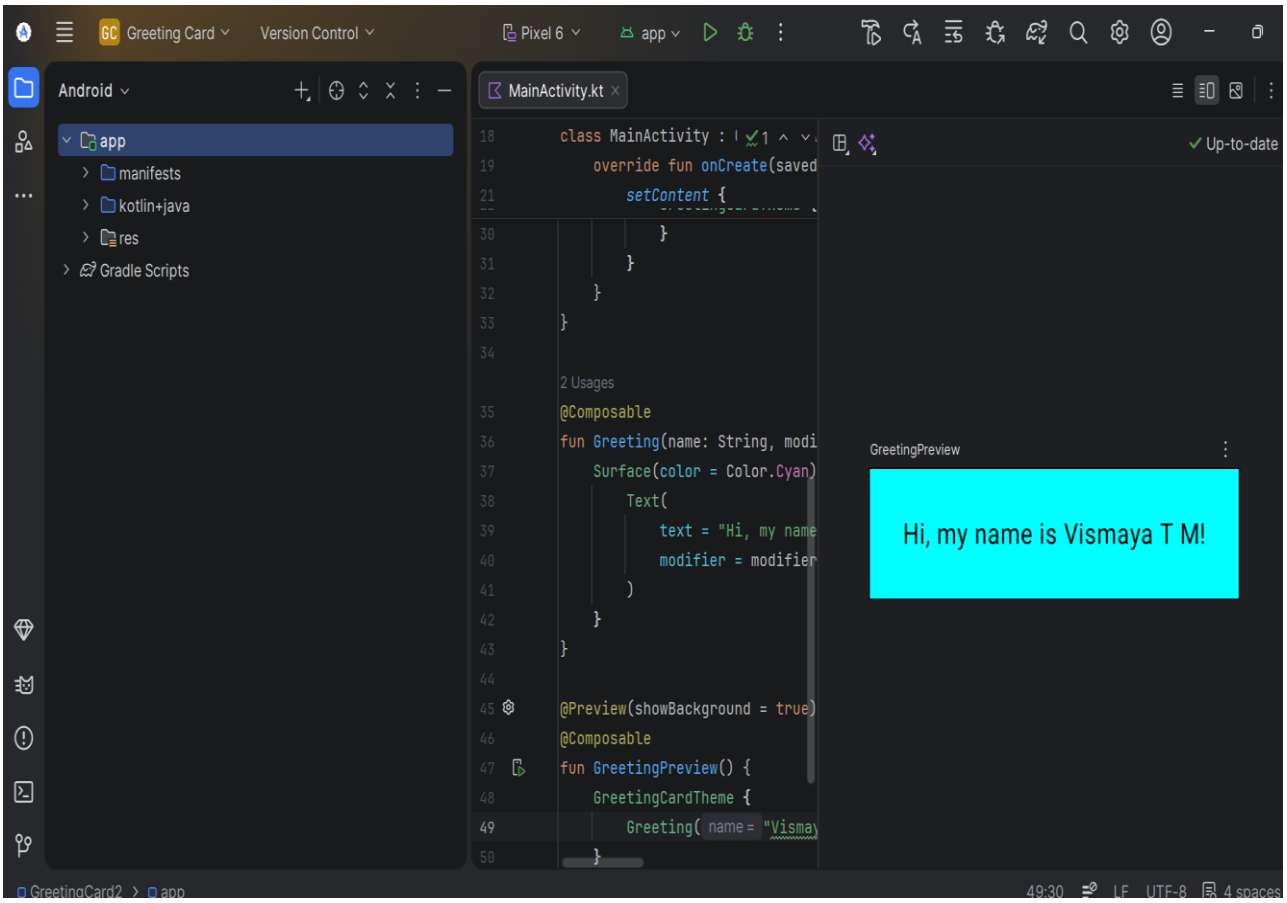
7. Problem-Solving Skills

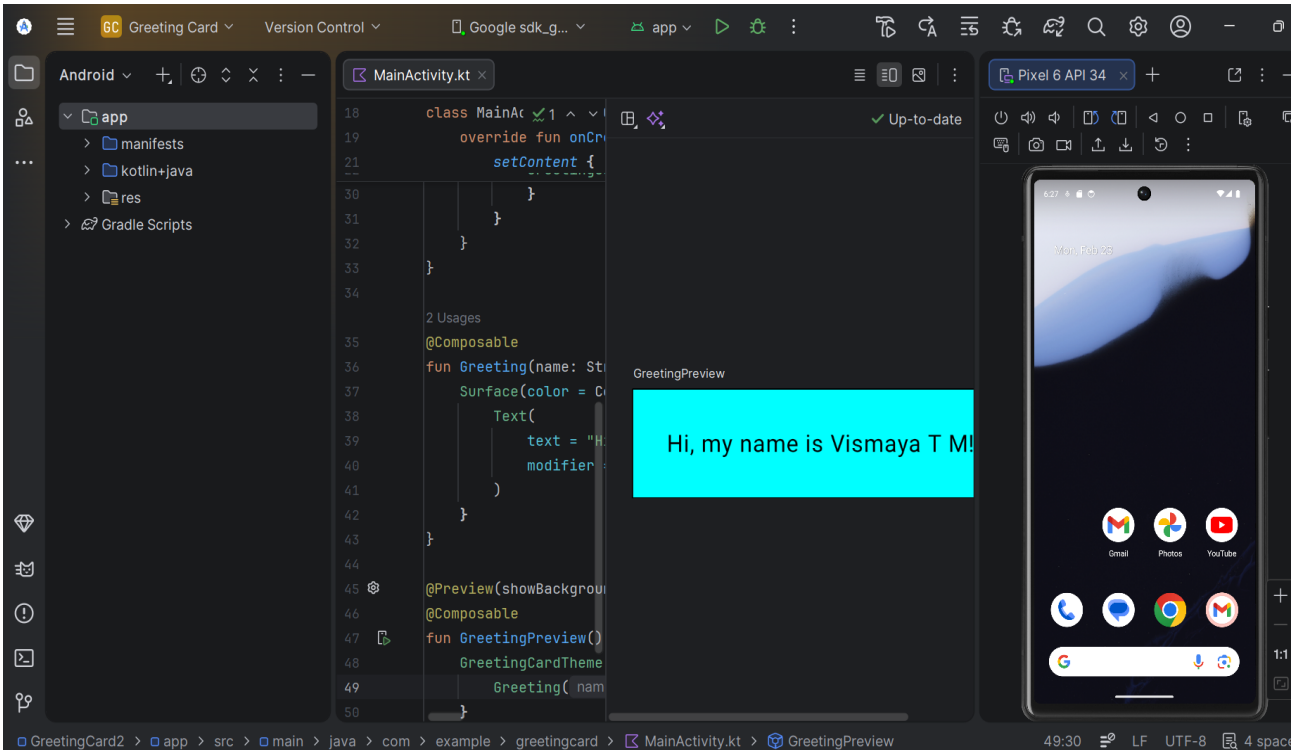
When my device was not detected, I identified the issue and solved it by changing USB settings. This improved my troubleshooting ability.

8. Confidence in Android Development

Completing this setup successfully gave me confidence in starting Android app development.




Greeting Card
Version Control
Pixel 6
app
Android
app
manifests
kotlin+java
res
Gradle Scripts
MainActivity.kt
class MainActivity :
override fun onCreate(saved
setContent {
}
}
}
}
2 Usages
@Composable
fun Greeting(name: String, modi
Surface(color = Color.Cyan)
Text(
text = "Hi, my name
modifier = modifier
)
}
}
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
GreetingCardTheme {
Greeting(name = "Vismay
}
Up-to-date
GreetingPreview
Hi, my name is Vismaya T M!
GreetingCard2 > app
49:30
LF
UTF-8
4 spaces


Greeting Card
Version Control
Google sdk_g...
app
Android
app
manifests
kotlin+java
res
Gradle Scripts
MainActivity.kt
class MainA
override fun onCr
setContent {
}
}
}
}
2 Usages
@Composable
fun Greeting(name: St
Surface(color = C
Text(
text = "H
modifier
)
}
}
@Preview(showBackgrou
@Composable
fun GreetingPreview()
GreetingCardTheme
Greeting(nam
}
Up-to-date
GreetingPreview
Hi, my name is Vismaya T M!
Pixel 6 API 34
Gmail
Photos
YouTube
GreetingCard2 > app > src > main > java > com > example > greetingcard > MainActivity.kt > GreetingPreview
49:30
LF
UTF-8
4 space

How Compose is started:

```
setContent {
  GreetingCardTheme {
    Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
      Greeting(
        name = "Nadiya",
        modifier = Modifier.padding(innerPadding)
      )
    }
  }
}
```

From this code, I understood that Jetpack Compose allows UI to be built directly in Kotlin. The setContent {} block replaces the traditional XML layout system. Instead of defining layouts separately, we define them using composable functions.

I also learned that UI elements are structured hierarchically, where one composable function can call another to build the complete screen.

- ComponentActivity is the base activity class for Compose apps.
- setContent {} defines the UI content.
- Scaffold provides a basic layout structure.
- Modifier is used to control size, padding, and layout behavior.

TASK – 2

Jetpack Compose is a modern UI toolkit used to build Android applications using Kotlin. Instead of XML layouts, Compose uses composable functions to create UI components. Layout primitives like Column, Row, and Box help structure the screen. Understanding when to use each primitive is important for designing clean and responsive interfaces.

1. Column

Column is a layout primitive in Jetpack Compose that arranges UI elements vertically from top to bottom.

Typical Use-Cases (2 examples):

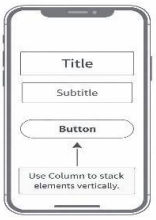
- Login screen (Title → Email → Password → Button)
- Profile screen (Image → Name → Bio)

One Limitation:

Column does not scroll by default. If content exceeds screen height, we must use verticalScroll().

This wireframe represents a simple vertical UI layout.

The Title, Subtitle, and Button are arranged from top to bottom using a Column composable. Column is used here because it stacks elements vertically in an organized manner. This layout is commonly used in login screens and greeting cards.



2. Box

Box is a layout primitive that stacks UI elements on top of each other.

Typical Use-Cases:

- Image with text overlay
- Notification badge on icon
- Centering a card in the screen (like my greeting project)

In my greeting card project, I used Box to center the card on the screen and to layer the gradient background behind the text.

Limitation:

Since Box overlays elements, incorrect alignment can cause UI overlapping issues.

Example UI: Profile image with small notification badge on top corner.

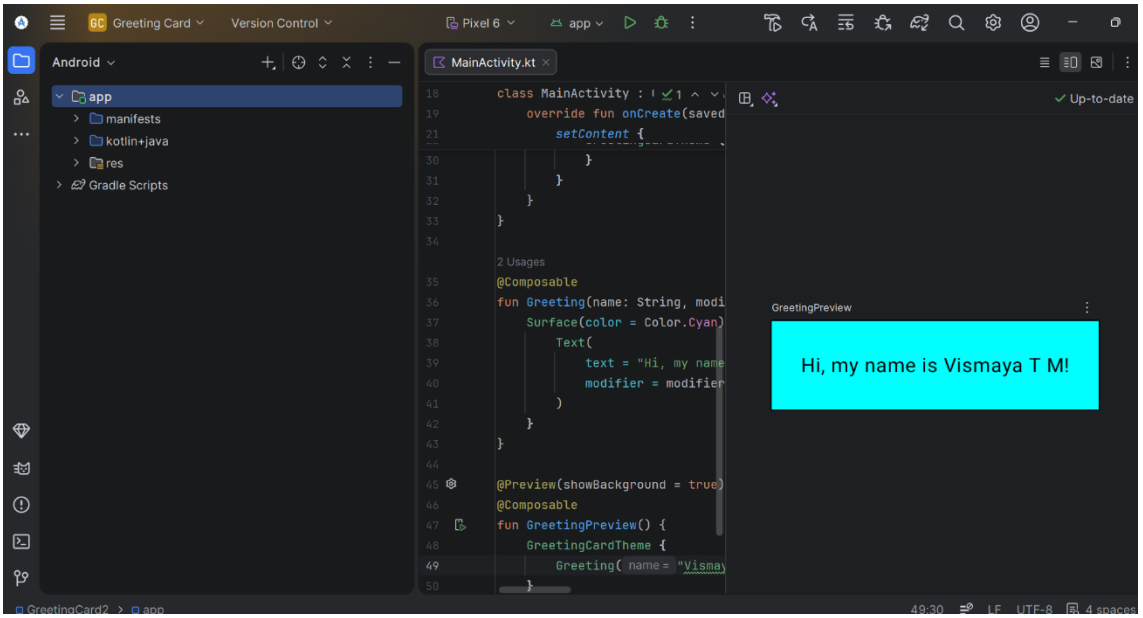
This wireframe shows a profile image with a small notification badge placed on the top-right corner. A Box composable is used here because it allows elements to be layered on top of each other. The badge overlaps the profile image, which is possible using Box alignment properties.

Comparison Table

Primitive	Direction	Best For	Limitation
Column	Vertical	Forms, stacked text	Needs scroll for long content
Row	Horizontal	Toolbars, button groups	May overflow width
Box	Overlay	Centering, badges, layering	Can overlap incorrectly

Optional Mini Experiment

I verified the use of Box in my Greeting Card project using Compose Preview and device testing.



In my Greeting Card project, I used the Box composable to center the greeting card on the screen. Box allows elements to be stacked and aligned easily. I used contentAlignment = Alignment.Center to position the card in the middle of the screen.

Box was also useful to manage background and UI layering properly. This helped me understand how Box controls alignment and overlapping behavior in Jetpack Compose.

Task 3 – Modifiers, Spacing & Responsive Behavior

This task explores how Jetpack Compose modifiers affect layout, spacing, and responsiveness. I experimented with three modifiers in my Greeting Card project to understand how padding, size, alignment, and arrangement impact UI on different screen sizes. Screenshots from Compose Preview and emulator were captured to document the behavior.

Pick 3 modifiers

In my code I have already used 3 modifiers

1. fillMaxSize() – used in the outer Box and Scaffold.
2. padding(innerPadding) – applied in Greeting function.
3. background(gradientBrush) – applied to the inner Box.

3 experiments

Experiment 1: fillMaxSize()

- **Expectation (1 line):** The Box should expand to occupy all available space in the parent container.
- **Observation:** The Box fills the whole screen and centers the Surface inside it.
- **Explanation:** fillMaxSize() makes the container responsive to different screen sizes. Changing screen size will keep the box filling the space and keep content centered.
- **Accessibility note:** Works well; content is centered and readable.

Experiment 2: padding(innerPadding)

- **Expectation (1 line):** Content inside Greeting should not overlap system bars and Scaffold insets.
- **Observation:** The greeting card stays inside safe bounds on all devices.
- **Explanation:** padding(innerPadding) ensures consistent spacing with system bars and app scaffolds. The layout respects edges, making it more responsive.
- **Accessibility note:** Safe spacing improves touch targets and prevents overlap.

Experiment 3: background(gradientBrush)

- **Expectation (1 line):** Card should have a vertical gradient background from light blue to dark blue.
- **Observation:** Gradient is applied correctly and fills the Surface.
- **Explanation:** Modifier order matters: .background() applied after .fillMaxSize() ensures the gradient covers the full card.
- **Accessibility note:** Colors have good contrast with text, ensuring readability.

Responsive Tips

1. Use fillMaxWidth() or fillMaxSize() to make containers adapt to screen size.
2. Use weight() for proportional widths/heights in Row or Column.
3. Use Arrangement and Alignment to position elements consistently across devices.

Accessibility Note

- For Surface width = 280.dp and height = 160.dp, small phones may make touch targets tight.
- **Fix:** Use Modifier.fillMaxWidth(0.8f) instead of fixed width to adapt proportionally.

Task 4 — Layout Critique & Design System Starter

App: Weather App (The Weather Channel)

Screen: Home screen showing current weather, hourly forecast, and daily forecast.

Structured Critique

Aspect	Observation	Notes / Issues
Layout Clarity	Current temperature and location are prominent at the top. Hourly forecast is in a horizontal row; daily forecast listed below.	Clear hierarchy; user can quickly find current weather and weekly forecast.
Spacing	Some elements are very close together (e.g., hourly forecast numbers and icons).	Suggest consistent spacing, especially for touchable areas.
Readability	Large font for current temp (20°) is readable; smaller fonts for hourly forecast may be hard to read on small devices.	Increase font size slightly for better visibility.
Navigation / Buttons	Minimal navigation; small icon at bottom right for menu.	Consider larger touch target for accessibility.
Accessibility	Contrast between background sky and white text is okay but could be stronger in bright conditions.	Add subtle shadow behind text or slightly darker overlay to improve contrast.
Performance / Complexity	Simple visuals; no heavy graphics.	Fine for smooth performance.

Proposed Improvements

1. Add **consistent 8dp spacing** between elements to prevent clutter.
2. Increase font size of hourly forecast to **14–16sp** for readability.
3. Improve text contrast for small icons and numbers (e.g., slight shadow or darker overlay).
4. Make bottom navigation icon touch target **48dp** for accessibility.
5. Use subtle card or background behind hourly and daily forecast for visual separation.
6. Standardize padding around daily forecast rows for consistent alignment.

Design System Starter Sheet

Category	Token / Value	Compose Usage Example
Spacing Scale	4dp / 8dp / 16dp / 24dp	Modifier.padding(8.dp)
Typography	Title / Current Temp: 24–28sp BoldHourly Forecast: 14–16sp RegularDaily Forecast: 16–18sp Regular	Text("20°", style = TextStyle(fontSize = 28.sp, fontWeight = FontWeight.Bold))
Colors	Primary: #2196F3 (sky blue)OnPrimary: #FFFFFF (text/icons)Background: #87CEEB (light blue)Surface: #FFFFFFError: #FF5252	color = Color(0xFF2196F3)
Buttons / Icons	Padding: 12dp vertical / 12dp horizontalCorner radius: 8–12dpMinimum touch: 48dp	IconButton(modifier = Modifier.size(48.dp))
Cards / Forecast Rows	Elevation: 4dpCorner radius: 12dpPadding inside: 8–12dp	Card(modifier = Modifier.padding(8.dp), shape = RoundedCornerShape(12.dp), elevation = 4.dp)
Inputs / Search Bar	Padding: 12dpFont size: 14–16sp	TextField(modifier = Modifier.padding(12.dp), textStyle = TextStyle(fontSize = 14.sp))

- The critique shows that current weather is easy to read and understand.
- Minor spacing, contrast, and touch target adjustments improve usability and accessibility.
- The Design System Starter provides reusable spacing, typography, color, and component rules for implementing consistent Compose layouts in future apps.

Conclusion

During this internship, I learned how to set up Android Studio and run apps on a physical device, which improved my confidence and problem-solving skills. Working with Jetpack Compose helped me understand how UI can be built directly in Kotlin using composable functions and layout primitives like Column, Row, and Box. Through modifier experiments, I saw how properties like padding, fillMaxSize, and background affect layout and responsiveness across different screen sizes. The layout critique and design system starter taught me the importance of spacing, typography, accessibility, and consistent design for a better user experience. Overall, these tasks strengthened my Android development skills and gave me a solid foundation to build responsive and user-friendly apps.

Submitted by: Vismaya T M

MindMatrix Internship - Android App Development using Gen AI

February 2026